

On the Infeasibility of Constructing Lenstra–Pomerance Counterexamples to Agrawal’s Conjecture via Twin-Smooth Subset Products

Dan Shumow

Claude (Anthropic Fable 5)

August 24, 2026

Abstract

Agrawal’s conjecture asserts that for r prime, $r \nmid n$, the congruence $(X - 1)^n \equiv X^n - 1 \pmod{n, X^r - 1}$ forces n prime or $n^2 \equiv 1 \pmod{r}$; if true it yields a deterministic $\tilde{O}(\log^3 n)$ primality test. The only known route to a counterexample is the Lenstra–Pomerance construction: a squarefree $n = p_1 \cdots p_k$ that is simultaneously Carmichael and Lucas–Carmichael, with side conditions, at a size heuristics place at hundreds of digits or (far) beyond — out of reach of search, but potentially in reach of *construction*. We report on a systematic attempt to execute that construction as a two-stage pipeline: harvest a pool of admissible primes (twin-smooth integers with prime sum in a fixed congruence class), then solve a two-sided 0/1 subset-product problem over $(\mathbb{Z}/\Lambda_-)^\times \times (\mathbb{Z}/\Lambda_+)^\times$. The outcome is asymmetric. The *supply* stage is solved: a partition-first harvest sidesteps a measured combinatorial obstruction (a universal ≈ 0.06 survivors-per-demand-bit invariant of partition-agnostic twin corpora, measured up to the complete $B = 547$ corpus of 82,026,426 pairs) and delivers validated pool-to-demand ratios of 13.8–44, an over-provisioning of one to two orders of magnitude. The *solver* stage is blocked: generic exact and heuristic solvers cap two to three orders of magnitude below the required odd-part dimension, and the one known scalable method — the subexponential ω -guided subset-product reduction of Alford, Grantham, Hayman, and Shallue (AGHS) — requires pool primes with high *identity density* on the group components, which we show is unobtainable here: under the double Carmichael + Lucas–Carmichael condition, harvestable pools have identity density ≈ 0.003 – 0.008 (about $100\times$ below AGHS pools), while the divisor-paradigm harvest that would supply AGHS-like density yields an essentially empty pool (0–2 primes) at all feasible scales. This quantified density gap is the negative result. We emphasize scope: this is not a disproof of Agrawal’s conjecture, nor a proof that no counterexample can be constructed; one escape — the full AGHS algorithm run on low-density pools — remains untested and is being pursued separately.

1 Introduction

For coprime integers n, r write $T(-1, n, r)$ for the congruence

$$(X - 1)^n \equiv X^n - 1 \pmod{n, X^r - 1}. \tag{1}$$

Conjecture 1.1 (Agrawal; Bhattacharjee–Pandey [2], popularized in the AKS paper [1]). *If r is a prime not dividing n and $T(-1, n, r)$ holds, then n is prime or $n^2 \equiv 1 \pmod{r}$.*

If Conjecture 1.1 were true, a single evaluation of (1) at one $r = O(\log n)$ would certify primality, giving a deterministic $\tilde{O}(\log^3 n)$ test, versus $\tilde{O}(\log^6 n)$ for the best proven algorithm [7]. Its status

is strange: Lenstra and Pomerance [3] gave a heuristic argument for $\gg x^{1-\varepsilon}$ counterexamples below x , yet the conjecture has survived exhaustive search to 10^{17} (Primaboinca, 2010–2020) and folklore places the least counterexample at hundreds of digits. The only concrete counterexample route — the Lenstra–Pomerance (L–P) construction (Theorem 2.1 below), extended by Popovych [4] and Hegde–Devaraj [6] — demands an n that is simultaneously a Carmichael and a Lucas–Carmichael number with additional congruence conditions; no such number is known, and none exists below 10^{18} [8].

The constructive program. Since any counterexample refutes the conjecture, minimality can be abandoned. The L–P density heuristic *improves* with size, and the construction is deterministic: when its arithmetic side conditions hold, (1) follows with no further luck required. This suggests a pipeline in the style of the large-scale Carmichael constructions of Alford–Granville–Pomerance [9] and Löh–Niebuhr [10]: **harvest** a pool $\mathcal{P}$ of primes satisfying the local conditions — which turn out to be exactly the *twin smooth integers with prime sum* of isogeny-based cryptography [14, 15, 16, 18, 20] in a fixed congruence class — and then **solve** for a subset $S \subseteq \mathcal{P}$ whose product satisfies a two-sided congruence, a 0/1 subset-product problem in the group $(\mathbb{Z}/\Lambda_-)^\times \times (\mathbb{Z}/\Lambda_+)^\times$ (Proposition 2.3). This paper reports what happened when that pipeline was built and measured end to end.

Contributions.

- (i) **A validated, over-provisioned supply method** (§3). We first measure a universal obstruction to mining existing twin-smooth corpora: across bounds $B = 100$ –547 and pool sizes 78 to 73,163 (the complete $B = 547$ corpus of 82,026,426 pairs), the best side-partition retains only ≈ 0.06 pool elements per bit of group demand — a factor 16–25 below feasibility. We then show that a *partition-first* harvest sidesteps this invariant entirely, with measured pool-to-demand ratios of 13.8–44 under a yield model validated (conservatively) against the complete $B = 547$ corpus.
- (ii) **Identification and quantification of a solver-stage obstruction** (§4). The subset-product problem splits into an easy 2-part ($\approx 16\%$ of demand bits, solved exactly at full scale) and a dominant odd part ($\approx 84\%$) on which generic exact and heuristic solvers cap two to three orders of magnitude short. The scalable method — the AGHS ω -guided reduction [11], which built a Carmichael number with 10,333,229,505 prime factors — needs pools of high *identity density* (≈ 0.3 –0.5). We measure that the double Carmichael + Lucas–Carmichael condition forces a dichotomy: harvestable pools have density ≈ 0.003 –0.008, and the divisor-paradigm harvest that would achieve AGHS-like density is empty (0–2 primes) at every feasible scale. The pipeline is blocked at the solver by this quantified *identity-density gap*.
- (iii) **Reusable machinery and diagnostics** (§6): a validated twin-smooth yield model, a near-complete C implementation of the CHM closure, a solver-frontier map for the double-condition subset-product problem, and the density gap itself as a diagnostic applicable to other Carmichael-type constructions under a double side condition.

What is not claimed. This paper does **not** disprove Agrawal’s conjecture, which remains open (and widely believed false). It does **not** prove that no counterexample can ever be constructed, nor even that the L–P construction is infeasible in principle. It establishes, quantitatively, that one specific and natural pipeline — twin-smooth harvest followed by subset-product solve, with the known solver families — is blocked at the solver stage, and it locates the blockage precisely (an identity-density gap of about two orders of magnitude). One escape is identified and untested; see §5, which should be read as part of the result, not as a disclaimer.

Provenance of numbers. Every quantitative claim below is a *measurement* produced by a script in the companion repository, recorded in its findings log; Appendix B maps each claim to the script and data file that produced it. Throughout we distinguish *proved* statements (the lemmas of §2), *measured* quantities (ratios, densities, solver frontiers), and *heuristic or conjectural* inputs (the L–P counterexample-density heuristic, Dickman-type yield extrapolations). The “blocked” conclusion is an empirical and structural argument, not a theorem.

2 The construction and its reduction

The statements in this section are proved in the companion working paper [23]; we restate them without proof, except where the proof is one line.

Theorem 2.1 (Lenstra–Pomerance [3]; statement following Váňa [5], who proves the $k \equiv 3 \pmod{4}$ case). *Let $n = p_1 \cdots p_k$ with $p_1, \dots, p_k$ distinct primes such that*

- (a) k is odd;
- (b) $p_i \equiv 3 \pmod{80}$ for all i ;
- (c) $p_i - 1 \mid n - 1$ for all i (n is Carmichael);
- (d) $p_i + 1 \mid n + 1$ for all i (n is Lucas–Carmichael).

Then $T(-1, n, 5)$ holds and $n^2 \not\equiv 1 \pmod{5}$; in particular such an n with $k \geq 3$ is a counterexample to Conjecture 1.1.

The entire difficulty of satisfying (c) and (d) simultaneously is concentrated in the following trivial but decisive observation.

Lemma 2.2 (Side-coprimality). *If n satisfies (c) and (d), then $\gcd(p_i - 1, p_j + 1) \mid 2$ for all i, j (including $i = j$). Consequently the odd primes dividing $\prod_i (p_i - 1)$ and those dividing $\prod_i (p_i + 1)$ form disjoint sets.*

Proof. An odd prime q dividing $p_i - 1$ and $p_j + 1$ divides both $n - 1$ and $n + 1$, hence their difference 2, a contradiction. $\square$

Under condition (b) the 2-adic and mod-5 parts of (c),(d) are automatic [23, Lem. 2.5], so the problem lives entirely in odd moduli prime to 5, split by Lemma 2.2 into two disjoint prime universes — a “minus side” supporting the $p_i - 1$ and a “plus side” supporting the $p_i + 1$.

2.1 Reduction to a two-sided subset-product problem

Fix disjoint finite sets Q_-, Q_+ of odd primes with $5 \notin Q_- \cup Q_+$, and define the *pool*

$$\mathcal{P} = \left\{ p \text{ prime} : p \equiv 3 \pmod{80}, \frac{p-1}{2} \text{ is } Q_- \text{-smooth}, \frac{p+1}{4} \text{ is } Q_+ \text{-smooth} \right\}.$$

Let $\Lambda_- = 2 \operatorname{lcm}\{p - 1 : p \in \mathcal{P}\}$ and $\Lambda_+ = 4 \operatorname{lcm}\{p + 1 : p \in \mathcal{P}\}$.

Proposition 2.3 ([23, Prop. 3.1]). *Let $S \subseteq \mathcal{P}$ with $|S|$ odd, $|S| \geq 3$, and $n = \prod_{p \in S} p$. If*

$$n \equiv 1 \pmod{\Lambda_-} \quad \text{and} \quad n \equiv -1 \pmod{\Lambda_+}, \tag{2}$$

then n satisfies conditions (a)–(d) of Theorem 2.1, hence is a counterexample to Conjecture 1.1.

Condition (2) is stronger than (c),(d) — it imposes the full lcm rather than the lcm over S — but in exchange it is linear: taking discrete logarithms in each cyclic component of $G = (\mathbb{Z}/\Lambda_-)^\times \times (\mathbb{Z}/\Lambda_+)^\times$ (Pohlig–Hellman [12] is cheap, all component orders being smooth by design), (2) becomes a 0/1 subset-sum over $\bigoplus_i \mathbb{Z}/m_i$, with one extra $\mathbb{Z}/2$ coordinate enforcing $|S|$ odd. The heuristic solvability condition is $2^{|\mathcal{P}|} \gg |G|$, i.e. $|\mathcal{P}| \gtrsim \log_2 \Lambda_- + \log_2 \Lambda_+$ (the *demand bits*) with slack for the solver.

2.2 The twin-smooth dictionary

Lemma 2.4 (Dictionary; [23, Lem. 4.1]). *Let $m \geq 1$ and $p = 2m + 1$. Then $p \in \mathcal{P}$ (for $Q_\pm$ large enough to contain the relevant primes) if and only if*

- (i) *m and $m + 1$ are both smooth with respect to $Q_- \cup Q_+ \cup \{2\}$, with the odd part of m being Q_- -smooth and the odd part of $m + 1$ being Q_+ -smooth;*
- (ii) *$m \equiv 1 \pmod{40}$;*
- (iii) *p is prime.*

Moreover (ii) alone implies $p \equiv 3 \pmod{80}$, $v_2(p - 1) = 1$, $v_2(p + 1) = 2$, and $5 \nmid m(m + 1)$.

Pairs of consecutive smooth integers $(m, m + 1)$ — *twin smooths* — with $2m + 1$ prime are precisely the parameters sought for the isogeny-based schemes B-SIDH [14] and SQIsign, and 2020–2025 produced an industrial literature on generating them: PTE sieving [15, 19], the optimized Conrey–Holmström–McLaughlin (CHM) iteration [17, 16] (whose $B = 547$ run yields 82,026,426 pairs), Pell/Størmer complete sets [21, 22, 18], and lattice-based record hunts [20]. By Lemma 2.4, a congruence slice of any twin-smooth dataset, filtered for primality of $2m + 1$, is a ready-made pool — before the one condition that literature never needs: the *global* side-partition forced by Lemma 2.2.

2.3 Pipeline vocabulary

The repository catalogs the pipeline components in a registry (`doc/registry.md`) whose identifiers we adopt: harvesters **H1–H8** (H1 native sieve; H2/H3 CHM closure in Python/C; H4 PTE import; H5 corpus streaming; **H6** partition-first; **H7** co-designed partition-first; **H8** divisor-paradigm), solvers **S1–S6** (S1 partition/coloring optimizer; **S2** GF(2) 2-part elimination; S3 annealing; S4 exact meet-in-the-middle; S5 per-prime reduction scaffold; **S6** AGHS ω -guided reduction), and validators V1–V4. The narrative of §§3–4 is: mining (H1–H5 $\rightarrow$ S1) fails by a measured invariant; partition-first harvest (H6/H7) solves supply; solvers S2–S4 hit a dimensional frontier; the scalable S6 needs a pool structure (H8) that the double condition makes unharvestable.

3 Supply is solved

This section establishes the positive half: pool supply is not the binding constraint. All quantities are measurements; scripts and data files are listed in Appendix B.

3.1 Mining fixed corpora is dead: the 0.06 invariant

The obvious first move is to mine the existing twin-smooth corpora: take all twins $(m, m + 1)$ below a smoothness bound B , keep the dictionary slice ($m \equiv 1 \pmod{40}$, $2m + 1$ prime), and then choose the side-partition $Q_- \sqcup Q_+$ maximizing the number of *surviving* elements — those whose m -support

lies entirely in Q_- and $(m + 1)/2$ -support in Q_+ . Feasibility requires survivors comparable to the demand bits of the group they generate.

Finding 3.1 (The 0.06 mining invariant — measured). Across seven experiments spanning $B = 100$ –547, truncated and complete corpora, and pool sizes 78 to 73,163, the optimized partition retains

$$\frac{\text{\#survivors}}{\text{\#demand bits}} \approx 0.060\text{--}0.083,$$

with element-greedy optimization and direct partition-space local search agreeing (Table 1). Feasibility needs this ratio $\gtrsim 1$; the shortfall is a factor 16–25.

The measurement includes the *complete* $B = 547$ corpus of 82,026,426 pairs (shared by M. Naehrig; 642,495 pairs with $m \equiv 1 \pmod{40}$, of which 73,163 have $2m + 1$ prime): the best partition found keeps 82 survivors against 984 demand bits (ratio 0.083). The invariant is thus measured, not extrapolated, on the entire published supply. The mechanism is visible in the data: partition-agnostic twins carry a mean total of ≈ 15 distinct odd primes across both sides at $B = 547$, so each kept element “spends” roughly two fresh primes (≈ 15 bits of new demand) that no later element may straddle. Optimization beats a random partition by 3.4–4.4 $\times$, but no optimizer can beat the arithmetic: **dataset mining is dead as a primary route**.

3.2 Partition-first harvesting (H6) and its gates

The invariant is a property of *partition-agnostic* supply. Inverting the order — fix the partition *first*, then harvest only compatible elements — removes it by construction. Concretely (harvester H6): choose $Q_+ =$ the odd primes $\leq B$ (except 5) and $Q_- =$ a band of medium primes in $(B, B']$; enumerate m as products of j primes of Q_- restricted to $m \equiv 1 \pmod{40}$; per candidate, trial-divide $(m + 1)/2$ over Q_+ and test $p = 2m + 1$ for primality. Every hit is compatible with the partition by construction; the cost moves from a global combinatorial obstruction into per-candidate yield, a local Dickman-type probability.

Two pre-registered gates governed the program. **G1** (yield exists): some measured cell must show pool/demand ≥ 0.5 at prototype scale with a fitted extrapolation ≥ 2.5 . **G2** (production spec): a frozen specification with projected pool/demand ≥ 2.5 and ≥ 3000 usable elements. Both passed:

Finding 3.2 (Partition-first supply — measured). On the measured $j = 4$ grid (Table 2), pool-to-demand ratios reach 13.8 at $(B, B') = (1259, 12590)$ — the frozen production specification: $Q_+ =$ odd primes ≤ 1259 ($\neq 5$), $Q_- \subset (1259, 12590]$, $j = 4$, projected pool $\approx 2.5 \times 10^5$ elements of mean 50 bits against $\approx 18,100$ demand bits — and up to 41–45.5 at larger B' . Partition-first thus beats the mining invariant by a factor of roughly 200–700 in the best cells.

Finding 3.3 (Model validation — measured, conservative). The sampling-based yield model behind Table 2 was validated out-of-sample against the complete $B = 547$ corpus (exhaustive twin population to 122 bits): across four asymmetric splits it predicted 0.41–1.04 $\times$ the true pool counts (geometric mean 0.68) — i.e. the model is conservative, never optimistic, and the contrast with the mining route (≈ 300 –500 $\times$ separation) is measured at scale on both sides with the same congruence, filters, and optimizer.

3.3 Corpus and closure validation

Supporting the supply claims, the repository’s C implementation of the CHM closure (H3, `chm_closure.c`) was validated against ground truth: it recovers 13,333 of the complete 13,374

pairs at $B = 100$ (99.7%), 33,118 at $B = 113$ (99.65% of the complete 33,233), and 346,110 at $B = 200$ — 99.98% of the originally published 346,192 and 99.2% of the true complete count 348,840 (the published figure was itself 0.8% short), with zero false positives and exact agreement on the largest pair in each case. The anchored counts $N(100) = 13,374$, $N(200) = 348,840$, $N(547) = 82,026,426$ fit $N(B) \sim B^\alpha$ with $\alpha \approx 4.7\text{--}5.4$, so fixed- B supply, while finite (Størmer [21, 22]), grows steeply in B .

Conclusion of §3. Supply is abundant, validated, and over-provisioned by one to two orders of magnitude relative to the heuristic feasibility threshold. Whatever blocks the pipeline is downstream.

4 The solver obstruction

By Proposition 2.3 the remaining task is: given the pool, find S with $|S|$ odd and $\prod_{p \in S} p \equiv (1, -1)$ in $G = (\mathbb{Z}/\Lambda_-)^\times \times (\mathbb{Z}/\Lambda_+)^\times$. Via CRT, G decomposes into cyclic prime-power components $\mathbb{Z}/\ell^a$ arising from the prime-power parts of each $q - 1$, $q \in Q_- \cup Q_+$; Pohlig–Hellman discrete logarithms turn the target into a 0/1 subset-sum, NP-hard in general and tractable only through structure. With pool size $N \gg \dim G$, solutions are heuristically super-dense; *finding* a 0/1 one is the crux.

4.1 The 2-part is easy — and small

Every component reduced mod 2 is linear over GF(2) (the unknowns being 0/1), so the 2-part of the system, plus the parity row, solves by Gaussian elimination. This scales without difficulty:

Finding 4.1 (Measured). At the frozen specification $(B, B') = (1259, 12590)$ the group has 4586 cyclic components and the full GF(2) system solves in 12 seconds (solver S2). But the 2-part accounts for only $\approx 16\%$ of the demand bits.

This overturned the program’s working assumption that the 2-torsion was the bulk of the problem. The remaining $\approx 84\%$ — $\approx 15,000$ bits at the frozen spec — lives in odd-prime components with primes ℓ up to 6269.

4.2 The odd part defeats the generic solvers

Two generic attacks on the odd part were built and pushed to their limits on faithful synthetic instances with planted solutions (Table 3): simulated annealing inside the GF(2) solution family (S3) stalls at ≥ 19 odd components, and exact meet-in-the-middle (S4) solves what annealing cannot ($N=40$, 15 odd components, in 11s) but is exponential in N , capping near $N \approx 40$ pool elements, ≈ 15 odd components, ≈ 40 odd bits. (Wagner’s generalized-birthday algorithm [13], the other natural generic candidate, needs list sizes exponential in the odd bit-count in its classic form and was not competitive in this regime.)

A harvest/solver co-design (H7) attacks the odd part from the supply side: constraining both factor bases to primes q whose $q - 1$ has odd part t_0 -smooth caps the largest odd prime in the group. Measured co-designed cells (Table 4) show a viable window — at $t_0 = 100$, $B = 2003$, $B' = 60090$ the harvest ratio stays 7.7 (above the G2 gate) while the odd support shrinks from 336 primes ≤ 6269 to 24 primes ≤ 97 . But co-design caps the odd prime *size*, not the odd *dimension*: recovering yield requires many primes, and the odd part remains at $\approx 24,000$ bits across thousands of components — two to three orders of magnitude beyond the S3/S4 frontier. Squeezing the dimension itself would require near-Fermat (2-power-heavy) primes, which are too sparse to harvest: a genuine yield/solver tension.

Finding 4.2 (Measured). Generic exact and heuristic solvers for the odd part cap at ≈ 15 odd components / ≈ 40 odd bits, versus $\approx 24,000$ odd bits (thousands of components) required at the co-designed specification — a shortfall of two to three orders of magnitude that co-design provably (by measurement) does not close.

4.3 The scalable method and what it requires

The frontier of Finding 4.2 is not the end of the story, because subset-products $\equiv 1$ in $(\mathbb{Z}/\Lambda)^\times$ at vastly larger scales *have* been solved: Alford, Grantham, Hayman, and Shallue [11], building on Löh–Niebuhr [10] and the AGP machinery [9], constructed a Carmichael number with 10,333,229,505 prime factors. Their method is not linear algebra: it is a subexponential ω -guided reduction, driving a distance-to-identity potential (the highest CRT component at which a partial product differs from the identity) to zero top-down, with a birthday step. It has no small-odd-part requirement, which is precisely why the generic-solver frontier above is a statement about the wrong solver family rather than about the problem.

The method does, however, have a structural input requirement. AGHS pools consist of primes p with $p - 1 \mid \Lambda$ for a fixed highly composite Λ : such a p is $\equiv 1$ on *every* component whose prime power divides $p - 1$, i.e. the pool vectors are identity on a large fraction of the components. Call this fraction the *identity density* of a pool element. High identity density is what makes ω -reduction move: most steps must leave most components untouched. AGHS-style pools have identity density ≈ 0.3 – 0.5 . Our partition-first pools are the opposite extreme: $p - 1 = 2m$ with m a product of $j \approx 4$ medium primes is identity on only $\approx j$ of the components touched by the pool — measured density ≈ 0.003 – 0.008 , about $100\times$ lower.

4.4 The identity-density gap (the core negative result)

If the double condition admitted an AGHS-style harvest, the pipeline would be back on track. The natural design (harvester H8, the *divisor paradigm*) fixes disjoint factor bases T_-, T_+ (side-coprimality), sets Λ_-, Λ_+ highly composite over them, and harvests pool primes $p = 2m + 1$ with m a (squarefree) divisor of $\prod T_-$, $m \equiv 1 \pmod{40}$, $2m + 1$ prime, and $(m + 1)/2$ squarefree over T_+ — exactly the $p - 1 \mid \Lambda_-, p + 1 \mid \Lambda_+$ structure the ω -solver wants, on both sides at once.

Finding 4.3 (N1: the divisor paradigm is unharvestable — measured). Across all feasible scales tested (Table 5), the H8 pool contains 0–2 primes. Small T_- (needed for decent density $\omega(m)/|T_-|$) yields an empty or near-empty pool; enlarging T_- collapses the density ($0.178 \rightarrow 0.067$ – 0.088) while the pool stays at 1–2. The root cause is structural: the double condition — *both* $p - 1 \mid \Lambda_-$ *and* $p + 1 \mid \Lambda_+$ for fixed moduli — is the product of two rare events, far more restrictive than the single condition $p - 1 \mid \Lambda$ that AGHS harvests satisfy.

Finding 4.4 (The identity-density gap — measured). Under the double Carmichael + Lucas–Carmichael condition:

pool	identity density	pool size
AGHS-style ($p - 1 \mid \Lambda$, single condition)	≈ 0.3 – 0.5	large (their regime)
partition-first H7 (harvestable)	≈ 0.003 – 0.008	$\approx 10^5$ (over-provisioned)
divisor-paradigm H8 (AGHS density)	≈ 0.07 – 0.18 where nonempty	0–2 (unharvestable)

One can have a harvestable pool *or* AGHS-like identity density — not both. The scalable solver family is therefore not straightforwardly applicable, and no other family in the measured frontier (Table 3) reaches the required dimension. That is the obstruction.

This is the punchline of the paper. The supply half of the pipeline is solved and over-provisioned (§3); the solve half is blocked, not by an absence of solvers, but by a quantified mismatch — roughly two orders of magnitude in identity density — between what the scalable solver needs and what the double condition can supply.

5 Scope, caveats, and the untested escape

This section bounds the result. It is part of the result.

1. **Not a disproof of Agrawal’s conjecture.** The conjecture remains open and is believed false; nothing here bears on its truth. A reviewer’s first question — “did you prove the conjecture has no counterexample?” — has answer *no*, emphatically.
2. **Not a proof that no counterexample is constructible.** The blockage established is of *this pipeline* (twin-smooth harvest $\rightarrow$ subset-product solve) with the *known solver families* (linear-algebraic, annealing, meet-in-the-middle, and ω -guided reduction as characterized by its identity-density requirement). Other constructions, and other solvers, are unconstrained by our measurements.
3. **Finite-scale and squarefree caveats on N1.** Finding 4.3 tested squarefree divisors at finite scales (Table 5). Prime-power moduli $\Lambda_{\pm}$ are untested, though the structural analysis points the same way (high density requires m to retain full prime powers, forcing m large); enormous scales are untested, and the measured density trend is against them.
4. **The one open escape: the full AGHS solver on low-density pools.** Whether the complete AGHS algorithm — two potentials, birthday step, non-uniformity exploitation — tolerates the ≈ 0.003 – 0.008 identity density of harvestable H7 pools is *untested*. The repository’s ω -reducer (`a8.omega.solver.py`) is a first strict top-down version that solves only the easy regime ($r \leq 20$ components at high identity density) and is too weak to settle the question. This escape is being pursued separately from this write-up; a positive outcome there would reopen the pipeline and supersede the negative conclusion for route $H7 \rightarrow S6$.
5. **Other unexhausted avenues.** A solver purpose-built for low-density many-prime pools; a different AKS modulus r (the group and congruence classes change with r , and some $r \neq 5$ may admit denser pools or a friendlier group); and the admissible-cell question of [23, Q. 6.2] — the boost obstruction [23, Prop. 6.1] shows the mod-80 cell is unreachable by the twin-smooth boosting family $2x^n - 1$ [16, 19], but whether *other* local-condition cells admit the L–P mechanism (and are reachable by structurally side-separated families) is open, and a positive answer would restructure the harvest entirely.

6 Reusable contributions and conclusion

Beyond the negative result, the program leaves behind machinery of independent use:

- **A validated twin-smooth supply model.** The partition-first yield model, validated conservative (geometric mean $0.68\times$, never optimistic) against the complete $B = 547$ corpus, plus

the near-complete C implementation of the CHM closure (99.2–99.98% of the exact complete sets, zero false positives) — including the observation that the published $N(200) = 346,192$ was itself 0.8% short of the true 348,840.

- **The 0.06 mining invariant** (Finding 3.1): a sharp, reproducible measurement of the global side-partition cost for partition-agnostic twin corpora, and its complete bypass by partition-first harvesting — relevant to any application that must multiplicatively combine twins under a side constraint.
- **A solver-frontier map** for two-sided 0/1 subset-product problems (Table 3), with a synthetic-instance regression harness (planted solutions, tunable identity density ρ) against which any new solver — in particular a full AGHS implementation — can be tested down to the harvestable regime $\rho \approx 0.004$.
- **The identity-density gap as a diagnostic.** For any Carmichael-type constructive problem under a *double* side condition, the two measurable quantities — harvest yield of divisor-structured pools, and identity density of harvestable pools — locate the pipeline on the harvestable/solvable dichotomy of Finding 4.4 before major compute is spent.

Conclusion. The Lenstra–Pomerance construction reduces cleanly to a two-sided subset-product problem whose supply side is, contrary to initial expectation, solved: partition-first harvesting overprovisions the pool by one to two orders of magnitude. The pipeline is blocked at the solver stage by a quantified identity-density gap: harvestable pools are $\approx 100\times$ below the density the scalable ω -guided solver family exploits, and the harvest that would supply that density is empty under the double Carmichael + Lucas–Carmichael condition. One escape — the full AGHS algorithm on low-density pools — remains open and under separate investigation. Until it is settled, the honest status of the constructive route to refuting Agrawal’s conjecture via twin smooths is: *supply yes, solver no, and the wall is measured.*

A Data tables

All tables reproduce measurements from the repository findings log `data/FINDINGS.md`; the generating script and log section are noted under each table.

B	corpus	pool	optimized survivors	surv/demand-bits
113	sieve, $X \leq 10^8$	59	8	0.061
113	sieve, $X \leq 10^{10}$	73	8	0.061
113	CHM closure (complete-ish)	78	8	0.061
257	sieve, $X \leq 10^9$	497	21	0.061
547	sieve, $X \leq 10^9$	2,715	51	0.068
200	CHM closure (near-complete)	562	17 / 16	0.060 / 0.061
547	complete corpus (82,026,426 pairs)	73,163	82	0.083

Table 1: The 0.06 mining invariant (Finding 3.1). “Pool” = twins with $m \equiv 1 \pmod{40}$ and $2m+1$ prime; survivors under the best side-partition found (element-greedy; for $B = 200$ also direct partition-space local search, second figure). Scripts: `phase1a_calibration.py`, `chm_closure.c`, `b547_pool_analysis.py`; FINDINGS “Calibration runs”, “Compiled CHM port”, “Definitive $B = 547$ coloring measurement”.

B	B'	ratio	pool	demand bits	rel. err
1259	12590	13.8	2.5×10^5	18,117	$\pm 24\%$
1259	18885	41.0	1.1×10^6	27,101	$\pm 26\%$
2003	20030	45.5	1.3×10^6	28,768	$\pm 24\%$
2003	40060	209	1.2×10^7	57,514	$\pm 32\%$

Table 2: Partition-first (H6) measured pool/demand ratios, reliable $j = 4$ cells (Finding 3.2); the (1259, 12590) row is the frozen specification. Script: `a2_grid.py` (data `a2_grid.json`); FINDINGS “A2: model validated ... G2 passed”.

solver	method	reaches	caps at
S2	GF(2) elimination (2-part + parity)	4586 components, 12 s	2-part only ($\approx 16\%$ of bits)
S3	annealing in GF(2) solution family	—	stalls ≥ 19 odd components
S4	exact meet-in-the-middle	$N=40$, 15 odd comps, 11 s	≈ 40 odd bits (exp. in N)
required (co-designed spec)		$\approx 24,000$ odd bits, thousands of odd components	

Table 3: The generic solver frontier on the odd part (Finding 4.2), on faithful synthetic instances with planted solutions. Scripts: `a4_solver.py`, `a6_oddsolver.py`; FINDINGS “A4 solver, G3 phase-1” and “A6 small-prime odd-part solver”.

B Reproducibility

The companion repository (PRIMES-Conjectures) contains every script and committed data file behind the claims of this paper. The findings log `data/FINDINGS.md` is the single source of truth for all quantities; the map below traces each claim to its generator.

claim	script (code/)	data (data/generated/ or log)
machinery unit tests (T1–T4)	<code>verify_machinery.py</code>	FINDINGS “Phase 0”
0.06 invariant, $B \leq 257$	<code>phase1a_calibration.py</code>	FINDINGS “Calibration runs”
CHM closure (C), $B \leq 200$ anchors	<code>chm_closure.c</code>	<code>chm_B200.log</code>
complete $B = 547$ pool + coloring	<code>b547_pool_analysis.py</code>	<code>b547_pool_stats.json</code>
A1 yield grid (G1)	<code>a1_yield_prototype.py</code>	<code>a1_yield_grid.json</code>
A2 grid (G2)	<code>a2_grid.py</code>	<code>a2_grid.json</code>
A2 out-of-sample validation	<code>a2_validate_and_extend.py</code>	<code>a2_validation.json</code>
group decomposition, GF(2) solve	<code>a4_solver.py</code>	FINDINGS “A4 solver”
co-design characterization/yield	<code>a5_codesign.py</code> , <code>a5_yield.py</code>	<code>a5_codesign_yield.json</code>
odd-part solver frontier	<code>a6_oddsolver.py</code>	FINDINGS “A6”
ω -reducer (partial S6)	<code>a7_lohniebuhr.py</code> , <code>a8_omega_solver.py</code>	FINDINGS “A7/A8”
N1 divisor harvest	<code>a9_divisor_harvest.py</code>	FINDINGS “N1”

All runs are deterministic (fixed seeds). The complete $B = 547$ twin corpus is private (courtesy of M. Naehrig) and is *not* in the repository; only aggregate statistics derived from it are committed (`b547_pool_stats.json`), which suffices to check every number quoted here.

References

- [1] M. Agrawal, N. Kayal, N. Saxena, *PRIMES is in P*, Ann. of Math. **160** (2004), 781–793.
- [2] R. Bhattacharjee, P. Pandey, *Primality testing*, BTech thesis, IIT Kanpur, 2001.

t_0	B	B'	ratio	max odd ℓ	# distinct odd
50	1259	40060	1.1	47	14
100	1259	40060	2.6	97	24
100	2003	60090	7.7	97	24
200	1259	40060	14.5	199	45
50	631	40060	0.0	47	14

Table 4: Co-designed harvest (H7): measured yield with both factor bases constrained to odd part of $q - 1$ being t_0 -smooth (hit counts low, $\pm 40\%$). Bold row: the viable window. Scripts: `a5_codesign.py`, `a5_yield.py` (data `a5_codesign_yield.json`); FINDINGS “A5 harvest/solver co-design”.

T_- bound	T_+ range	$ T_- + T_+ $	pool	mean density	max density
90	97–220	45	1	0.178	0.178
120	127–300	60	1	0.067	0.067
150	160–360	68	2	0.074	0.088
60	67–160	34	0	—	—

Table 5: N1: divisor-paradigm harvest (H8) yield and identity density under the double condition (Finding 4.3). Script: `a9_divisor_harvest.py`; FINDINGS “N1: divisor-paradigm harvest (H8) is unharvestable”.

- [3] H. W. Lenstra, Jr., C. Pomerance, *Remarks on Agrawal’s conjecture*, AIM workshop notes, 2003. <https://aimath.org/WWN/primesinp/articles/html/50a/>
- [4] R. Popovych, *A note on Agrawal conjecture*, Cryptology ePrint 2009/008.
- [5] T. Vána, *Agrawal’s conjecture and Carmichael numbers*, Comenius University Bratislava, 2009.
- [6] A. Hegde, P. Devaraj, *Heuristics for the construction of counterexamples to the Agrawal conjecture*, Springer PROMS **344**, 2021.
- [7] H. W. Lenstra, Jr., C. Pomerance, *Primality testing with Gaussian periods*, J. Eur. Math. Soc. **21** (2019), 1229–1269.
- [8] R. G. E. Pinch, *The Carmichael numbers up to 10^{18}* , arXiv:math/0604376.
- [9] W. R. Alford, A. Granville, C. Pomerance, *There are infinitely many Carmichael numbers*, Ann. of Math. **139** (1994), 703–722.
- [10] G. Löh, W. Niebuhr, *A new algorithm for constructing large Carmichael numbers*, Math. Comp. **65** (1996), 823–836.
- [11] W. R. Alford, J. Grantham, S. Hayman, A. Shallue, *Constructing Carmichael numbers through improved subset-product algorithms*, Math. Comp. **83** (2014), 899–915; arXiv:1203.6664.
- [12] S. Pohlig, M. Hellman, *An improved algorithm for computing logarithms over $GF(p)$* , IEEE Trans. Inf. Theory **24** (1978), 106–110.
- [13] D. Wagner, *A generalized birthday problem*, CRYPTO 2002.
- [14] C. Costello, *B-SIDH: supersingular isogeny Diffie–Hellman using twisted torsion*, ASIACRYPT 2020.
- [15] C. Costello, M. Meyer, M. Naehrig, *Sieving for twin smooth integers with solutions to the Prouhet–Tarry–Escott problem*, EUROCRYPT 2021; ePrint 2020/1283.
- [16] G. Bruno, M. Corte-Real Santos, C. Costello, J. K. Eriksen, M. Meyer, M. Naehrig, B. Sterner, *Cryptographic smooth neighbors*, ASIACRYPT 2023; ePrint 2022/1439.

pool structure	identity density	harvestable under double condition?
AGHS ($p - 1 \mid \Lambda$, highly composite Λ)	$\approx 0.3\text{--}0.5$	no (Table 5: pool 0–2)
H7 ($p - 1 = 2m$, m a product of $j \approx 4$ medium primes)	$\approx 0.003\text{--}0.008$	yes (pool $\approx 10^5$)

Table 6: The identity-density gap (Finding 4.4): a $\approx 100\times$ mismatch between what the scalable ω -solver exploits and what the double condition can supply. FINDINGS “N1”, subsection “The identity-density gap”.

- [17] J. B. Conrey, M. A. Holmström, T. L. McLaughlin, *Smooth neighbors*, Experiment. Math. **22** (2013), 195–202.
- [18] J. Buzek, J. Hasan, J. Liu, M. Naehrig, A. Vigil, *Finding twin smooth integers by solving Pell equations*, arXiv:2211.04315.
- [19] B. Sterner, *Towards optimally small smoothness bounds for cryptographic-sized twin smooth integers and their isogeny-based applications*, SAC 2024; ePrint 2023/1576.
- [20] E. Mulder, B. Sterner, W. van Woerden, *Large smooth twins from short lattice vectors*, ANTS XVII; ePrint 2025/1462.
- [21] C. Størmer, *Quelques théorèmes sur l’équation de Pell $x^2 - Dy^2 = \pm 1$ et leurs applications*, Skr. Norske Vid. Akad., 1897.
- [22] D. H. Lehmer, *On a problem of Størmer*, Illinois J. Math. **8** (1964), 57–79.
- [23] D. Shumow and Claude, *Toward a counterexample to Agrawal’s conjecture via twin smooth integers*, working draft v0.1, August 2026 (companion repository, `doc/working-paper.tex`).